

05 Modelo de Datos

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

05 Modelo de Datos

4

title: "Cartridge & Cloud — Modelo de Datos" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" application_version_reference: "0.0.26" persistence_schema_reference: 2	5
Cartridge & Cloud — Modelo de Datos	5
0. Propósito, alcance y autoridad	5
1. Principios del modelo	6
1.1. Una sola fuente de verdad	6
1.2. Definición separada de instancia	6
1.3. Estado explícito	7
1.4. Mutaciones validadas y atómicas	7
1.5. Cantidades y dinero exactos	7
1.6. Datos derivados no duplicados	7
1.7. Referencias por ID, no por nombre visual	8
1.8. Colecciones deterministas	8
1.9. Fallo temprano en authoring	8
1.10. Compatibilidad evolutiva	8
2. Capas de datos y ciclo de vida	8
2.1. Clasificación principal	8
2.2. Fases de vida	9
2.3. Datos estáticos	9
2.4. Estado mutable	9
2.5. Historial y auditoría	9
3. Estrategia de identidad	10
3.1. Familias de ID	10
3.2. StableId	10
3.3. IDs de catálogo legibles	10
3.4. IDs de instancia	10
3.5. Igualdad y casing	11
3.6. IDs prohibidos	11
4. Ownership y fuentes de verdad	11
4.1. Matriz principal	11
4.2. Duplicaciones transitorias observadas	11
4.3. Resúmenes denormalizados	12
4.4. GameObjects y ownership	12
5. Mapa consolidado del modelo implementado	12
5.1. Agregados actuales	12
5.2. Dependencias conceptuales	12
5.3. Fronteras de agregado	13
6. Sesión, slots y metadatos	13
6.1. GameSession	13
6.2. GameSessionSnapshot	13
6.3. Slots	14
6.4. Metadatos recomendados	14
7. Grid, placement, acceso y espacio autorado	14
7.1. Value objects de grid	14
7.2. PlacedObjectRecord	14
7.3. Ocupación	15
7.4. Acceso	15
7.5. Arquitectura fija frente a objetos dinámicos	15
7.6. Deuda actual	16
8. Productos, categorías, tags y precios	16

8.1. ProductDefinition	16
8.2. Separación de responsabilidades	16
8.3. ProductSalePriceCatalog	16
8.4. Cotización congelada	16
8.5. Modelo objetivo de contenido	17
8.6. Familias iniciales	17
9. Inventario y conservación de unidades	17
9.1. Tipos principales	17
9.2. Modelo agregado	17
9.3. Transferencia atómica	18
9.4. Ecuación de conservación	18
9.5. Contenedores previstos	18
9.6. Persistencia	18
10. Proveedores, catálogos y pedidos	19
10.1. Proveedor	19
10.2. Pedido	19
10.3. Estados	19
10.4. Identidad e idempotencia	19
10.5. Persistencia actual	19
11. Entregas, cajas y recepción	20
11.1. Delivery	20
11.2. ShipmentBox	20
11.3. Commit de recepción	20
11.4. Persistencia objetivo	20
12. Displays, asignación y reposición	21
12.1. DisplayDefinition	21
12.2. DisplayInstance	21
12.3. Stock visible	21
12.4. RestockTask	21
12.5. Persistencia	22
12.6. Relación con placement	22
13. Clientes, perfiles y navegación	22
13.1. CustomerProfile	22
13.2. CustomerInstance	22
13.3. Navegación	23
13.4. Spawn	23
13.5. Persistencia	23
13.6. Modelo conceptual futuro	23
14. Shopping, reservas y carrito	23
14.1. ShoppingIntent	23
14.2. ShoppingReservation	24
14.3. Reserva lógica y stock físico	24
14.4. ShoppingCart	24
14.5. CustomerShoppingSession	24
14.6. Persistencia cruzada	25
15. Cola, estación y checkout	25
15.1. CheckoutQueueEntry	25
15.2. CheckoutQueue	25
15.3. CheckoutStation	26
15.4. CheckoutTransaction	26
15.5. Límite transaccional	26
15.6. Persistencia	26
16. Jornada, tiempo y cierre	26
16.1. StoreDay	26
16.2. Reglas temporales	27
16.3. Snapshot de cierre	27
16.4. Actividad diaria	27
16.5. Persistencia	27
17. Economía, precios y ledger	28

17.1. CurrencyCode	28
17.2. Money	28
17.3. EconomyLedger	28
17.4. Saldo	28
17.5. Resultado diario	28
17.6. Extensiones futuras	29
18. UI, tutorial, accesibilidad y proyecciones	29
18.1. Proyecciones UI	29
18.2. Navegación UI	29
18.3. Tutorial	29
18.4. Accesibilidad	29
19. Presentación representativa y catálogos visuales	30
19.1. Separación funcional/visual	30
19.2. Catálogo Phase 1	30
19.3. Phase1StoreState	30
19.4. Regla de authoring	30
20. Snapshot integrado schema 2	30
20.1. Cabecera	30
20.2. Subestados	31
20.3. Records persistentes	31
20.4. Validación cruzada	31
20.5. Campos ausentes que requieren decisión	32
21. Serialización, almacenamiento y recuperación	32
21.1. DTO sin Unity	32
21.2. Escritura segura	32
21.3. Estados de repositorio	32
21.4. Recovery	33
21.5. Equivalencia de round trip	33
22. Versionado y migraciones	33
22.1. Regla de schema	33
22.2. Migración incremental	33
22.3. Versiones futuras	33
22.4. Cambios de catálogo	33
22.5. Migración de las rutas actuales	34
23. Validación de contenido y authoring	34
23.1. Validaciones generales	34
23.2. Productos	34
23.3. Displays	34
23.4. Proveedores	34
23.5. Clientes	35
23.6. Herramientas	35
24. Datos derivados, caches y proyecciones	35
24.1. Derivados principales	35
24.2. Caches	35
24.3. Proyecciones históricas	35
25. Idempotencia, concurrencia lógica y límites transaccionales	35
25.1. Operaciones idempotentes	36
25.2. No hay concurrencia de threads como excusa	36
25.3. Preflight	36
25.4. Secuencias	36
26. Modelos diferidos cercanos	36
26.1. Empleados	36
26.2. Investigación	37
26.3. Puestos informáticos	37
27. Comercio online y logística	37
27.1. Entidades	37
27.2. Estados de pedido	38
27.3. Inventario compartido	38
27.4. Persistencia	38

28. Publishing	38
28.1. Entidades objetivo	38
28.2. Contratos e hitos	38
28.3. Due diligence	39
29. Desarrollo interno	39
30. Plataforma digital	39
31. Infraestructura y servicios online	40
32. Mercado y competidores	40
33. Reglas de extensión	40
33.1. Añadir una entidad	40
33.2. Añadir un campo persistente	41
33.3. Añadir un enum state	41
33.4. No reutilizar campos	41
34. Estrategia de pruebas del modelo	41
34.1. Value objects	41
34.2. Agregados	41
34.3. Servicios multiagregado	42
34.4. Snapshot	42
34.5. Catálogos	42
34.6. Migraciones	42
35. Deuda de datos y plan de convergencia	42
35.1. Orden recomendado	42
36. Criterios de aceptación del modelo	43
36.1. Definition of Done de una nueva entidad persistente	43
Anexo A. Inventario de tipos de dominio observados	43
Anexo B. Estados y enums implementados	47
Anexo C. Cardinalidades críticas	48
Anexo D. Diagrama de persistencia schema 2	49
Anexo E. Convenciones de naming	49
Anexo F. Glosario	49
Actualización del modelo W0	50
Estado operativo vigente tras W0	50
Entidades y campos requeridos	50
Regla de cierre y no propagación	50

05 Modelo de Datos

Metadatos normativos v2.0
document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE
lifecycle: LIVING DOCUMENT

title: "Cartridge & Cloud — Modelo de Datos" author: "VRM Games / Blas Luis Rocha González" date: "2026-07-07" lang: "es-ES" document_version: "2.0" status: "APPROVED / IMPLEMENTED / VALIDATED / CONSOLIDATED BASELINE" project: "Cartridge & Cloud" platform: "PC / Steam" engine: "Unity 6.3 LTS 6000.3.18f1" render_pipeline: "URP 17.3.0" application_version_reference: "0.0.26" persistence_schema_reference: 2

Cartridge & Cloud — Modelo de Datos

0. Propósito, alcance y autoridad

Este documento define el modelo de datos de *Cartridge & Cloud*: identidades, entidades, value objects, agregados, catálogos, estados, relaciones, invariantes, ownership, proyecciones, historial, persistencia, migraciones y reglas de extensión.

Su objetivo no es duplicar el GDD ni describir la implementación clase por clase como haría el TDD. Su función es establecer con precisión:

- qué información existe;
- qué información es estática, mutable, derivada, histórica o persistente;
- quién es propietario de cada dato;
- qué IDs identifican definiciones, instancias, operaciones y registros;
- qué relaciones son obligatorias;
- qué transiciones son válidas;
- qué invariantes deben mantenerse antes y después de cada operación;
- qué datos deben guardarse;
- qué datos deben reconstruirse;
- cómo se valida y migra un snapshot;
- cómo pueden añadirse sistemas futuros sin romper los IDs y contratos base.

El documento es normativo para nuevas entidades, nuevos snapshots, catálogos, servicios de aplicación y herramientas de authoring. Cuando una implementación contradiga este modelo se deberá decidir expresamente si existe un defecto, una deuda de migración o un cambio aprobado.

0.1. Jerarquía de autoridad

La consolidación utiliza el siguiente orden:

1. 00_Enfoque_y_Alcance.md, para pilares, límites y clasificación de alcance.
2. 01_Game_Design_Document.md, para reglas funcionales y comportamiento esperado.
3. 02_Vertical_Slice_Specification.md, para capacidades e invariantes obligatorias.
4. 03_Technical_Design_Document.md, para arquitectura, ownership y persistencia.
5. Modelo_de_Datos_v0.5, como autoridad sobre el modelo integrado en la baseline v0.6.
6. Modelo_de_Datos_v0.4, como fotografía de la transición tras Sprint 5.
7. Modelo_de_Datos_v0.3 de baseline v0.4, como fuente conceptual extensa.
8. La copia v0.3 de baseline v0.3, únicamente como trazabilidad histórica.

Las dos copias v0.3 tienen la misma estructura y el mismo contenido funcional. Solo difieren en fecha, motor validado y estado de Sprint 0. La copia de baseline v0.4 se toma como fuente conceptual de referencia.

0.2. Estados usados

Estado	Significado
IMPLEMENTADO	Existe una representación concreta en el código o snapshot actual.

Estado	Significado
OBJETIVO DEL SLICE	Debe existir para cerrar el vertical slice aunque la ruta actual sea transitorio.
DEUDA DE CONVERGENCIA	Existen dos representaciones o falta integrar un subestado.
DIFERIDO CERCANO	Modelo definido para empleados, investigación o puestos, pero no activo.
VISIÓN DE MEDIO PLAZO	Modelo de comercio online y logística.
VISIÓN TARDÍA	Publishing, desarrollo, plataforma, infraestructura o mercado.
DERIVADO	Se calcula desde datos autoritativos y normalmente no se persiste.
HISTÓRICO	Registro inmutable o append-only necesario para auditoría.
TRANSITORIO	Existe solo durante una operación y no debe sobrevivir a una carga.

0.3. Términos normativos

- **Debe:** requisito obligatorio.
- **No debe:** prohibición.
- **Puede:** opción compatible.
- **Debería:** recomendación fuerte que requiere justificación si se omite.
- **Fuente de verdad:** propietario autoritativo del dato.
- **Snapshot:** representación serializable coherente de un conjunto de estados.
- **Record:** DTO persistente sin comportamiento de dominio.
- **Catálogo:** conjunto estático e indexado de definiciones.
- **Instancia:** objeto mutable con identidad propia durante una partida.

1. Principios del modelo

1.1. Una sola fuente de verdad

Cada dato mutable debe tener un propietario inequívoco. La UI, vistas, GameObjects, builders, proyecciones y caches no pueden convertirse en fuentes paralelas.

Ejemplos:

- la cantidad de producto pertenece a InventoryContainer;
- una reserva pertenece a ShoppingReservationRegistry y referencia un display;
- la posición en cola pertenece a CheckoutQueue;
- el estado de la estación pertenece a CheckoutStation;
- los movimientos económicos pertenecen a EconomyLedger;
- la escena contiene representación y referencias, no saldos ni cantidades autoritativas;
- los ScriptableObject contienen definiciones, no progreso de partida.

1.2. Definición separada de instancia

Una definición describe una clase de contenido y se comparte entre partidas:

```
ProductDefinition
DisplayDefinition
SupplierDefinition
CustomerProfile
StoreDayPolicy
Phase1FurnitureDefinition
```

Una instancia describe un elemento concreto de una partida:

```
InventoryContainer
DisplayInstance
PurchaseOrder
Delivery
CustomerInstance
```

ShoppingReservation
CheckoutQueueEntry
CheckoutTransaction
StoreDay

Una definición no debe adquirir campos mutables como cantidad, posición de cola, estado de pedido o desgaste de una instancia. Una instancia referencia la definición por ID estable y no duplica sus datos salvo que se necesite congelar una condición histórica.

1.3. Estado explícito

Los ciclos importantes se representan mediante enums y transiciones, no mediante combinaciones de booleanos.

Ejemplos implementados:

PurchaseOrderStatus: Draft → Submitted → Delivered → Received
DeliveryStatus: AwaitingReceipt → PartiallyReceived → Received
ShoppingReservationState: Active → Released | Consumed
CustomerShoppingState: Searching → HoldingReservations → ReadyForCheckout
→ CheckedOut | Abandoned
CheckoutQueueEntryState: Waiting → Called → Processing → Completed | Cancelled
CheckoutTransactionState: Pending → Completed | Failed
StoreDayState: BeforeOpen → Open → Closing → Closed

Los booleanos derivados, como IsEmpty, IsActive o CanAcceptCustomers, pueden existir como consultas, pero no sustituyen al estado autoritativo.

1.4. Mutaciones validadas y atómicas

Toda mutación debe seguir:

entrada tipada
→ validación completa
→ cálculo del resultado
→ commit único
→ resultado tipado
→ proyección/feedback

Una operación fallida no debe dejar cantidades, reservas, dinero o estados parcialmente modificados. Las operaciones multiagregado requieren un servicio de aplicación que coordine preflight, commit, rollback lógico o idempotencia.

1.5. Cantidades y dinero exactos

- Las cantidades físicas usan enteros no negativos y el value object Quantity.
- Las capacidades usan enteros positivos.
- El dinero usa long en unidades menores y CurrencyCode de tres caracteres.
- No se usa float para saldos, costes, precios, impuestos o ingresos.
- Una operación entre monedas distintas debe rechazarse salvo que exista un sistema explícito de conversión.

1.6. Datos derivados no duplicados

No deben persistirse por defecto:

- AvailableCapacity, porque se deriva de capacidad y uso;
- VisibleUnitCount, porque se deriva del inventario y el límite visible;
- totales de una línea de pedido, porque se derivan de cantidad y coste;
- posición visual de un cliente sobre NavMesh;
- texto localizado;
- GameObjects, componentes o referencias de escena;
- caches de diccionarios reconstruibles;

- vistas del HUD;
- métricas que puedan recalcularse exactamente desde ledger e historial.

Puede persistirse un valor derivado cuando represente una decisión histórica, una cotización congelada o una auditoría. En ese caso debe indicarse su fuente y momento de cálculo.

1.7. Referencias por ID, no por nombre visual

Los nombres traducidos, nombres de GameObject, rutas de jerarquía y nombres de asset no se usan como claves de negocio. El cambio de un prefab, etiqueta o traducción no debe invalidar un save.

1.8. Colecciones deterministas

Los catálogos y snapshots deben tener orden estable cuando se serialicen o comparen. Los registros pueden ordenarse por ID ordinal, posición de cola, fecha o secuencia explícita. No se debe depender del orden accidental de un Dictionary o de una jerarquía Unity.

1.9. Fallo temprano en authoring

Los assets de catálogo deben validar:

- ID vacío o duplicado;
- referencia inexistente;
- capacidad no positiva;
- precio negativo o moneda incoherente;
- categoría no registrada;
- huella inválida;
- prefab ausente cuando sea obligatorio;
- clave de localización vacía;
- relación circular no permitida;
- estado inicial imposible.

1.10. Compatibilidad evolutiva

Los sistemas futuros deben añadir entidades y subestados sin cambiar el significado de IDs existentes. Una ampliación no puede reutilizar un campo antiguo con otra semántica. Cuando una entidad cambie de forma incompatible se incrementa schema y se define migración.

2. Capas de datos y ciclo de vida

2.1. Clasificación principal

Capa	Propósito	Ejemplos	Pers
Configuración	Reglas globales	duración del día, capacidad de cola, dificultad	No c
Catálogo estático	Definiciones de contenido	productos, displays, proveedores, perfiles	Ass
Estado de partida	Hechos mutables actuales	inventarios, pedidos, displays, clientes	Sí c
Estado transitorio	Operación en curso reconstruible	preview, hover, path temporal	No
Historial	Hechos confirmados	ledger, transacciones, resumen diario	Sí s
Snapshot	DTO coherente	IntegratedGameStateSnapshot	Sí
Proyección	Datos preparados para UI	HUD, paneles, descriptor de slot	Nor
Analítica	Métricas agregadas	conversión, espera, roturas de stock	Pre
Presentación	Mapping funcional→visual	catálogo de prefabs, iconos, materiales	Ass

Capa	Propósito	Ejemplos	Pers
------	-----------	----------	------

2.2. Fases de vida

Authoring

- importación/validación de catálogos
- composición runtime
- creación o restauración de partida
- mutaciones de dominio
- captura de snapshot
- serialización
- almacenamiento + backup
- carga + validación
- restauración
- reproyección visual

Cada fase debe tener contratos explícitos. Un `MonoBehaviour` no debe serializar directamente su estado privado para convertirse en la única representación persistente.

2.3. Datos estáticos

Los datos estáticos se distribuyen con el juego y se identifican mediante IDs estables. Se materializan desde `ScriptableObject` hacia modelos de dominio o registros de aplicación.

Catálogos observados:

- `ProductCatalogAsset;`
- `SupplierCatalogAsset;`
- `DisplayCatalogAsset;`
- `CustomerProfileCatalogAsset;`
- `ProductSalePriceCatalogAsset;`
- `StoreDaySettingsAsset;`
- `Phase1ContentCatalogAsset;`
- `Phase1PresentationCatalogAsset;`
- `RepresentativePrefabCatalogAsset;`
- `Phase1RuntimeAssetRegistryAsset.`

2.4. Estado mutable

El estado mutable debe existir en objetos de dominio y registros controlados. Los servicios de aplicación coordinan cambios entre agregados. La infraestructura guarda y recupera snapshots, pero no decide reglas de negocio.

2.5. Historial y auditoría

Los hechos que no deben reescribirse silenciosamente incluyen:

- ventas completadas;
- movimientos del ledger;
- recepción confirmada;
- cierre de jornada;
- migraciones aplicadas;
- recuperación desde backup;
- contratos empresariales futuros;
- liquidaciones y reembolsos.

La retención concreta puede compactarse para rendimiento, pero el resultado económico y la idempotencia deben conservarse.

3. Estrategia de identidad

3.1. Familias de ID

Familia	Uso	Formato recomendado
ID de catálogo	Definiciones distribuidas con el juego	slug ASCII estable y legible
ID de instancia	Objetos creados en una partida	GUID N o secuencia estable prefijada
ID de operación	pedidos, transacciones, postings	estable, único e idempotente
ID de sesión	partida lógica	StableId, GUID N de 32 caracteres
ID de slot	ubicación de guardado	entero 0..2
ID de escena/contexto	composición fija	slug estable, no nombre visual accidental
ID de localización	texto traducible	clave namespaced
ID histórico	hechos append-only	único y no reutilizable

3.2. StableId

StableId valida un GUID en formato N: 32 caracteres hexadecimales sin guiones. Se usa para la identidad de sesión y puede reutilizarse en nuevas instancias cuando convenga.

Ejemplo: 6f9619ff8b86d011b42d00cf4fc964ff

3.3. IDs de catálogo legibles

Los IDs estáticos deberían seguir:

```
product_game_nova_01
product_console_orbit_01
category_physical_game
display_wall_shelf_tier_e
supplier_general_01
customer_profile_budget
furniture_checkout_counter_tier_e
```

Reglas:

- minúsculas ASCII;
- snake_case;
- prefijo de dominio;
- sin espacios;
- sin texto traducido;
- no incluir versión visual del prefab;
- no reutilizar un ID retirado para otro concepto;
- renombrar solo mediante migración y alias documentado.

3.4. IDs de instancia

Ejemplos:

```
display:7b2...
order:000014
customer:day_0007:000023
reservation:...
checkout:...
ledger:...
```

El prefijo mejora diagnóstico, pero la unicidad no debe depender solo de él.

3.5. Igualdad y casing

Los IDs se comparan de forma ordinal y sensible a mayúsculas/minúsculas. La normalización se realiza al crear o importar el dato, no en cada consulta. El save no debe contener dos IDs que solo difieran por casing.

3.6. IDs prohibidos

No se permiten como identidad persistente:

- `GetInstanceID()` de Unity;
- nombre de `GameObject`;
- índice de hijo;
- ruta de escena;
- posición world-space;
- hash no estable entre ejecuciones;
- texto localizado;
- nombre visible del producto;
- referencia directa a prefab.

4. Ownership y fuentes de verdad

4.1. Matriz principal

Dato	Propietario autoritativo	Lectores/proyecciones	Persistencia
Sesión y slot	<code>GameSession</code> / raíz integrada	menú, autosave, UI	Sí
Día actual	<code>StoreDay</code> + coordinador de avance	HUD, resultados	Sí
Efectivo	economía/saldo integrado	HUD, compra, cierre	Sí
Producto estático	<code>ProductDefinitionRegistry</code>	inventario, displays, UI	No; se referen
Cantidad por contenedor	<code>InventoryContainer</code>	UI, displays, checkout	Sí
Pedido	<code>PurchaseOrder</code> / servicio de órdenes	proveedores, UI	Sí
Entrega y cajas	<code>Delivery</code>	recepción, UI	Sí mientras
Asignación de display	<code>DisplayInstance</code>	visual, reposición	Sí
Cliente	<code>CustomerInstanceRegistry</code>	vista y UI	Sí si se guar
Reserva	<code>ShoppingReservationRegistry</code>	carrito, checkout	Sí si está ac
Carrito/sesión	<code>CustomerShoppingSessionRegistry</code>	checkout, UI	Sí durante j
Cola	<code>CheckoutQueue</code>	estación, UI	Sí durante j
Estación	<code>CheckoutStation</code>	checkout, UI	Sí
Transacción	registry de checkout	ledger, resultados	Sí/histórico
Ledger	<code>EconomyLedger</code>	HUD, resultados, fiscalidad	Sí
Ocupación grid	<code>PlacementOccupancyRegistry</code>	preview, acceso, visual	Debe persis
Arquitectura fija	escena/contexto autorado	runtime y visual	No como m
Mapping visual	catálogos representativos	factory/builder	No como es

4.2. Duplicaciones transitorias observadas

Existen dos niveles de persistencia:

1. `GameSessionSnapshot` schema 1, con sesión, slot, fechas, día y efectivo.
2. `IntegratedGameStateSnapshot` schema 2, con el estado integrado del slice.

También existe `Phase1StoreState`, utilizado por la capa representativa, con secuencias, órdenes, stock, fixtures, ventas e importes acumulados.

Estas rutas son **deuda de convergencia**. El objetivo es una raíz persistente autoritativa o una composición de subestados claramente versionados. No se deben escribir dos archivos independientes para el mismo slot con saldos o días divergentes.

4.3. Resúmenes denormalizados

`CurrentDay` y `CashCents` pueden aparecer en metadatos de slot para lectura rápida, pero su valor debe proceder del estado integrado. Una proyección denormalizada debe reconstruirse o validarse contra la fuente antes de guardarse.

4.4. GameObjects y ownership

Un `GameObject` puede mostrar un display, cliente, puerta o mueble. No es propietario de:

- su ID funcional;
- la cantidad de producto;
- la reserva;
- el saldo;
- el estado persistente de la jornada.

La destrucción y recreación de una vista no debe eliminar el dato de dominio.

5. Mapa consolidado del modelo implementado

5.1. Agregados actuales

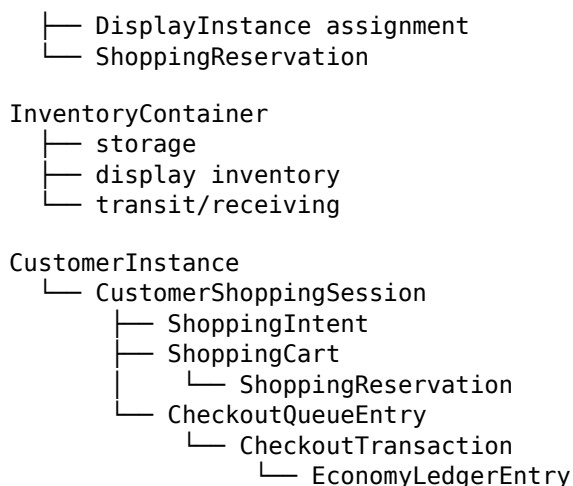
Agregado o raíz	Entidades/value objects relacionados	Estado
Sesión	<code>GameSession</code> , <code>StableId</code> , <code>SaveSlotId</code>	IMPLEMENTADO
Placement	<code>PlacedObjectRecord</code> , grid, ocupación y acceso	IMPLEMENTADO; persistencia in
Productos	definiciones, categorías, tags y registry	IMPLEMENTADO
Inventario	contenedores, stacks, quantity, transfer service	IMPLEMENTADO
Proveedores	supplier, catálogo y entries	IMPLEMENTADO
Pedido	<code>PurchaseOrder</code> , lines, status	IMPLEMENTADO
Entrega	<code>Delivery</code> , <code>ShipmentBox</code>	IMPLEMENTADO
Display	definición, instancia, asignación e inventario interno	IMPLEMENTADO
Cliente	profile, instance, navigation plan y registry	IMPLEMENTADO
Shopping	intent, reservation, cart, session y registries	IMPLEMENTADO
Checkout	queue, station, transaction y policy	IMPLEMENTADO
Jornada	<code>StoreDay</code> , policy, activity y closure snapshot	IMPLEMENTADO
Economía	Money, precios, ledger y resultado diario	IMPLEMENTADO
Persistencia	snapshot schema 2 y records	IMPLEMENTADO
Fase representativa	catálogos y <code>Phase1StoreState</code>	IMPLEMENTADO/TRANSITORIO

5.2. Dependencias conceptuales

```

ProductDefinition
├── SupplierCatalogEntry
├── InventoryStack
└── ProductSalePrice

```



5.3. Fronteras de agregado

- Un InventoryContainer protege su capacidad y cantidades.
- Una transferencia entre dos contenedores se coordina mediante servicio.
- Un DisplayInstance protege asignación e inventario de exposición.
- Una reserva protege ownership de unidades durante shopping.
- Un carrito no crea unidades; referencia reservas activas.
- El checkout coordina reservas, carrito, cola, estación, transacción y ledger.
- El cierre coordina clientes, cola, reservas, shopping y jornada.
- El snapshot valida relaciones cruzadas que un agregado aislado no puede comprobar.

6. Sesión, slots y metadatos

6.1. GameSession

Campos implementados:

Campo	Tipo	Regla
SessionId	StableId	GUID N válido, inmutable
SlotId	SaveSlotId	0, 1 o 2
CreatedUtc	DateTime	UTC, inmutable
CurrentDay	int	mínimo 1
CashCents	long	resumen económico transitorio/compatibilidad

GameSession no debe convertirse en un segundo ledger. El efectivo persistido debe converger con la economía integrada.

6.2. GameSessionSnapshot

Schema actual: 1.

Incluye CreatedUtc y UpdatedUtc, y exige:

- timestamps UTC;
- UpdatedUtc >= CreatedUtc;
- día mayor o igual a uno;
- schema soportado.

Esta estructura pertenece al save skeleton inicial. Su mantenimiento debe decidirse antes de crear un schema posterior.

6.3. Slots

SaveSlotId permite tres slots numerados 0..2. La UI los proyecta mediante SlotDescriptor:

- Empty;
- Ready;
- Recovered;
- Corrupt;
- UnsupportedSchema;
- StorageFailure.

Un descriptor es una proyección. No sustituye al snapshot.

6.4. Metadatos recomendados

El snapshot futuro puede separar un header mínimo:

```
SaveHeader
  schemaVersion
  applicationVersion
  sessionId
  slotId
  createdUtc
  updatedUtc
  currentDay
  cashMinorUnits
  currencyCode
  playTimeSeconds
  recoveryState
  contentManifestVersion
```

playTimeSeconds y contentManifestVersion son objetivos, no campos implementados en schema 2.

7. Grid, placement, acceso y espacio autorado

7.1. Value objects de grid

Tipo	Campos/semántica
GridCoordinate	X, Z; coordenada entera
GridSize	Width, Depth; ambos positivos
GridRotation	0°, 90°, 180°, 270°
GridFootprint	anchor, tamaño base, rotación y celdas ocupadas
GridBounds	límites permitidos
GridWorldPosition	conversión aplicación mundo/grid

La celda lógica vigente equivale a 0,5 m.

7.2. PlacedObjectRecord

Campos:

- PlacementInstanceId Id;

- string DefinitionId;
- GridCoordinate Anchor;
- GridRotation Rotation;
- GridSize BaseSize.

La huella efectiva se deriva de tamaño y rotación. No debe persistirse una lista redundante de celdas salvo que exista una razón de compatibilidad.

7.3. Ocupación

PlacementOccupancyRegistry es la fuente de verdad runtime de celdas ocupadas. Debe garantizar:

- no solapamiento;
- ID único;
- celdas dentro de límites;
- liberación exacta al retirar;
- consulta determinista;
- snapshot ordenado para persistencia o validación.

7.4. Acceso

Tipo	Función
AccessAnchorId	identidad estable de acceso obligatorio
AccessAnchor	ID + celda
StoreAccessLayout	conjunto de anchors y reglas
AccessValidationResult	éxito o razón de bloqueo

El acceso no debe deducirse de una puerta visual. La escena fija registra anchors funcionales.

7.5. Arquitectura fija frente a objetos dinámicos

La arquitectura de StoreInitial.unity:

- paredes;
- suelo;
- accesos;
- zonas fijas;
- contextos;
- colliders;
- referencias de navegación;

no se guarda como mobiliario dinámico. Se identifica mediante la versión o ID de la escena y se reconstruye al cargar.

Los objetos colocados por el jugador sí deben persistir:

```
PlacedObjectSaveRecord
  instanceId
  definitionId
  anchorX
  anchorZ
  rotation
  optionalStateReference
```


7.6. Deuda actual

IntegratedGameStateSnapshot schema 2 no contiene todavía placement. Phase1StoreState sí contiene Phase1PlacedFixtureRecord. Antes de cerrar la convergencia debe definirse una única representación persistente para mobiliario dinámico, sin guardar arquitectura fija como si fuera comprada o colocada.

8. Productos, categorías, tags y precios

8.1. ProductDefinition

Campos implementados:

Campo	Tipo	Regla
Id	ProductDefinitionId	estable y no vacío
DisplayNameKey	string	clave de localización
CategoryId	ProductCategoryId	categoría registrada
Tags	colección de ProductTagId	únicos, orden estable

La definición no contiene cantidad ni estado de venta.

8.2. Separación de responsabilidades

El coste de compra no pertenece necesariamente a ProductDefinition: depende del proveedor y su catálogo. El precio de venta pertenece al catálogo/política de precios. Esto permite:

- varios proveedores por producto;
- costes diferentes;
- promociones futuras;
- precios por tienda o canal;
- histórico de cotización;
- cambios de balance sin duplicar definiciones.

8.3. ProductSalePriceCatalog

- tiene una moneda única;
- una entrada por producto;
- todos los precios deben ser positivos;
- no admite productos duplicados;
- produce una cotización para checkout.

8.4. Cotización congelada

CheckoutQuote y CheckoutQuoteLine convierten carrito y catálogo en una cotización:

- reserva;
- producto;
- cantidad;
- precio unitario;
- total de línea;
- total general.

La cotización debe permanecer estable durante el commit. Un cambio de precio posterior no debe alterar una venta ya confirmada.

8.5. Modelo objetivo de contenido

Campos futuros compatibles:

```
ProductDefinition
  id
  displayNameKey
  descriptionKey
  categoryId
  tags[]
  physicalSizeClass
  weightClass
  fragilityClass
  releaseEra
  compatibilityTags[]
  defaultVisualId
  localizationNamespace
```

Los campos de balance por canal permanecen en catálogos independientes.

8.6. Familias iniciales

El contenido representativo vigente utiliza seis productos iniciales. Las familias conceptuales pueden incluir juegos físicos, consolas, mandos, auriculares y accesorios. La ampliación debe realizarse mediante definiciones, no mediante nuevos branches de código por producto.

9. Inventario y conservación de unidades

9.1. Tipos principales

Tipo	Responsabilidad
Quantity	cantidad entera no negativa/positiva según operación
InventoryCapacity	límite total en unidades
InventoryContainerId	identidad del contenedor
InventoryContainerType	Unspecified, Generic, Storage, Display, Transit
InventoryStack	producto + cantidad
InventoryContainer	colección de cantidades por producto
InventoryTransferService	transferencia atómica entre contenedores

9.2. Modelo agregado

Un contenedor mantiene un diccionario interno por ProductDefinitionId. UsedCapacity es la suma de cantidades y AvailableCapacity se deriva.

Invariantes:

- ninguna cantidad negativa;
- capacidad positiva;
- no almacenar un stack de cantidad cero salvo como ausencia;
- un producto solo aparece una vez por contenedor;
- UsedCapacity <= Capacity;
- el ID de contenedor es único;
- la cantidad solicitada debe ser positiva.

9.3. Transferencia atómica

```
preflight
  source != destination
  product exists
  source contains enough
  destination has capacity
→ remove source
→ add destination
→ success
```

Ante un fallo, la suma global no cambia. El servicio debe devolver una razón tipada:

- cantidad inválida;
- mismo contenedor;
- definición ausente;
- producto ausente en origen;
- cantidad insuficiente;
- capacidad insuficiente.

9.4. Ecuación de conservación

Para cada producto y ámbito de partida:

```
entradas confirmadas
- ventas consumidas
- descartes autorizados
- devoluciones a proveedor autorizadas
= unidades en contenedores
+ unidades reservadas lógicamente que siguen físicamente en display
+ unidades en tránsito explícito
```

Una reserva no duplica la unidad. La unidad permanece en el display hasta el commit o se modela con una transición explícita, pero nunca cuenta dos veces.

9.5. Contenedores previstos

- almacén;
- exposición;
- recepción/transit;
- contenedor técnico de caja si fuera necesario;
- picking online futuro;
- paquete futuro;
- devoluciones;
- dañados.

Cada nuevo tipo debe justificar reglas distintas. No se crea un enum por ubicación visual si el comportamiento es equivalente.

9.6. Persistencia

InventoryContainerSaveRecord contiene:

- ContainerId;
- Capacity;
- colección ProductQuantitySaveRecord.

El tipo de contenedor no aparece en el record schema 2. La restauración debe resolverlo por composición o catálogo. Si varios tipos comparten IDs o reglas distintas, deberá añadirse el tipo o una referencia de definición en un schema posterior.

10. Proveedores, catálogos y pedidos

10.1. Proveedor

SupplierDefinition contiene ID y clave de nombre. SupplierCatalog enlaza una definición con entradas de producto.

SupplierCatalogEntry contiene:

- ProductId;
- UnitCostCents;
- UnitsPerBox;
- MinimumBoxes;
- MaximumBoxes.

El coste es específico del proveedor.

10.2. Pedido

PurchaseOrder contiene:

- PurchaseOrderId;
- SupplierId;
- líneas ordenadas;
- estado;
- total de unidades;
- total de coste.

PurchaseOrderLine contiene:

- producto;
- número de cajas;
- unidades por caja;
- cantidad ordenada derivada;
- coste unitario;
- total derivado.

10.3. Estados

Draft → Submitted → Delivered → Received
└──────────→ Cancelled, según regla

Las transiciones inválidas no mutan.

10.4. Identidad e idempotencia

- Un pedido tiene ID único.
- Una línea se identifica por producto dentro del pedido o por ID explícito futuro.
- El mismo pedido no puede recibirse dos veces.
- La recepción parcial debe registrar progreso.
- El coste económico se publica una sola vez mediante posting key.

10.5. Persistencia actual

SupplierOrderSaveRecord está aplanado y contiene un producto por record:

- order ID;
- supplier ID;
- product ID;
- estado;

- unidades pedidas;
- unidades recibidas;
- coste unitario.

Para pedidos multilínea, la identidad del record debe definirse con claridad. Opciones:

1. PurchaseOrderSaveRecord + PurchaseOrderLineSaveRecord[];
2. records aplanados con clave compuesta (orderId, productId) y header separado.

La primera opción es más expresiva y evita repetir estado de cabecera.

11. Entregas, cajas y recepción

11.1. Delivery

Campos:

- DeliveryId;
- PurchaseOrderId;
- SupplierId;
- cajas;
- número recibido;
- estado.

Estados:

AwaitingReceipt → PartiallyReceived → Received

11.2. ShipmentBox

Cada caja tiene:

- ID;
- order ID;
- product ID;
- quantity;
- IsReceived.

La caja es la unidad de idempotencia de recepción. Volver a procesar una caja recibida debe fallar con BoxAlreadyReceived.

11.3. Commit de recepción

```
validar caja y pedido  
→ validar destino y capacidad  
→ validar coste/posting  
→ transferir o añadir unidades  
→ marcar caja recibida  
→ actualizar entrega/pedido  
→ publicar coste económico una sola vez
```

Si no puede completarse, no se marca la caja como recibida.

11.4. Persistencia objetivo

El schema integrado debe conservar entregas o suficiente información para reconstruirlas. No basta con el contador de unidades recibidas si existen cajas parciales, coste idempotente o representación física pendiente.

```
DeliverySaveRecord
  deliveryId
  orderId
  supplierId
  state
  boxes[]
```

```
ShipmentBoxSaveRecord
  boxId
  productId
  quantity
  isReceived
```

12. Displays, asignación y reposición

12.1. DisplayDefinition

Campos:

- ID;
- clave de nombre;
- capacidad;
- límite de unidades visibles;
- categorías admitidas;
- placement definition ID.

Reglas:

- capacidad positiva;
- visible limit entre 1 y capacidad;
- categorías sin duplicados;
- placement ID no vacío.

12.2. DisplayInstance

Contiene:

- ID de instancia;
- definición;
- inventario interno `display:{instanceId}`;
- asignación opcional de producto.

Una asignación solo puede cambiar cuando el inventario está vacío. El display puede admitir todas las categorías si la lista permitida está vacía.

12.3. Stock visible

`VisibleUnitCount = min(quantity assigned product, visibleUnitLimit)`

Es derivado y no se persiste.

12.4. RestockTask

Campos:

- task ID;
- contenedor origen;
- display destino;
- producto;

- cantidad solicitada;
- cantidad completada;
- estado.

Estados: Pending, Completed, Cancelled.

La tarea es un registro de intención/operación; la transferencia efectiva sigue siendo la fuente de verdad de cantidades.

12.5. Persistencia

DisplaySaveRecord contiene:

- display ID;
- definition ID;
- assigned product ID opcional;
- inventory container ID.

El inventario se guarda por separado y se valida que exista. La relación es 1:1.

12.6. Relación con placement

DisplayDefinition.PlacementDefinitionId conecta la definición funcional con la definición de colocación. Una instancia colocada debe poder enlazarse con DisplayInstanceId. Ese mapping debe persistirse o reconstruirse de forma determinista. No se debe buscar por nombre del prefab.

13. Clientes, perfiles y navegación

13.1. CustomerProfile

Campos implementados:

- ID;
- clave de nombre;
- peso de spawn;
- paciencia en segundos;
- número de paradas de browsing;
- velocidad de movimiento.

El perfil es estático. La instancia copia o referencia los valores necesarios.

13.2. CustomerInstance

Campos:

- customer instance ID;
- profile ID;
- navigation plan;
- state;
- current target index;
- remaining patience seconds.

Estados implementados:

WaitingToEnter → Entering → Browsing → Leaving → Despawned

Los estados de compra se modelan por separado en CustomerShoppingSession. Esto evita mezclar navegación física con negocio, pero requiere reglas de sincronización.

13.3. Navegación

CustomerNavigationPlan contiene targets identificados por CustomerNavigationPointId y tipo:

- Entry;
- Browse;
- Exit.

Cada target puede incluir dwell time. El path NavMesh concreto es transitorio y no se persiste.

13.4. Spawn

- CustomerSpawnPolicy: máximo activo e intervalo.
- CustomerArrivalClock: acumulación temporal determinista.
- CustomerSpawnQueue: solicitudes pendientes.
- CustomerSpawnRequest: IDs de request, instancia y profile + plan.

Las secuencias o semillas necesarias para reproducibilidad deben persistirse si una carga a mitad de jornada debe continuar exactamente.

13.5. Persistencia

CustomerSaveRecord contiene:

- customer ID;
- profile ID;
- estado;
- paciencia restante;
- índice de navegación.

No guarda posición world-space ni path. La restauración debe resolver una posición coherente por contexto y estado. Si la equivalencia espacial exacta es requisito, deberá añadirse un ancla lógica, no una referencia a Transform.

13.6. Modelo conceptual futuro

El GDD contempla presupuesto, sensibilidad al precio, afinidades, satisfacción y arquetipo. Estos campos deben añadirse a perfiles, intenciones o estado de sesión sin romper los IDs actuales.

```
CustomerProfile
  id
  spawnWeight
  basePatience
  budgetRange
  priceSensitivity
  categoryAffinities[]
  fallbackPolicy
  satisfactionWeights
```

14. Shopping, reservas y carrito

14.1. ShoppingIntent

Representa qué quiere intentar comprar un cliente:

- intent ID;
- customer ID;
- desired units;
- política de categorías/fallback.

No garantiza disponibilidad.

14.2. ShoppingReservation

Campos:

- reservation ID;
- customer ID;
- display ID;
- product ID;
- quantity;
- state.

Estados:

Active → Released

Active → Consumed

Consumed → Active (solo rollback controlado antes del commit final)

Invariantes:

- cantidad positiva;
- cliente existente;
- display existente;
- producto asignado compatible;
- stock activo reservado no supera on-hand;
- ownership único;
- una reserva consumida no se consume dos veces.

14.3. Reserva lógica y stock físico

La reserva no mueve físicamente la unidad por sí sola. Reduce la disponibilidad para otros clientes.

La fórmula es:

```
availableToReserve(display, product)
= onHand(display, product)
- activeReserved(display, product)
```

14.4. ShoppingCart

El carrito contiene líneas construidas desde reservas, no cantidades libres.

Campos:

- cart ID;
- customer ID;
- capacity units;
- líneas;
- total units derivado.

Una reserva no puede aparecer dos veces. Una reserva de otro cliente no puede añadirse.

14.5. CustomerShoppingSession

Relaciona:

- cliente;
- intent;
- carrito;
- estado.

Estados:

```
Searching
→ HoldingReservations
→ ReadyForCheckout
→ CheckedOut
↳ Abandoned
```

14.6. Persistencia cruzada

Schema 2 guarda:

- ShoppingSessionSaveRecord;
- ReservationSaveRecord;
- records de cola.

El snapshot valida que:

- cada sesión referencia un cliente existente;
- no hay dos sesiones para el mismo cliente;
- no hay dos sesiones para el mismo carrito;
- cada reserva referencia cliente, display y carrito compatibles;
- la reserva y la sesión comparten ownership.

15. Cola, estación y checkout

15.1. CheckoutQueueEntry

Campos:

- entry ID;
- customer ID;
- cart ID;
- state.

Estados:

```
Waiting → Called → Processing → Completed
                                └──→ Cancelled
```

15.2. CheckoutQueue

La cola mantiene:

- capacidad;
- colección ordenada activa;
- estado de aceptación/sealed;
- current entry.

Invariantes:

- FIFO;
- customer y cart no duplicados;
- posición contigua 1-based al persistir;
- solo el frente puede ser llamado;
- no se encola cuando está sellada;
- no se excede capacidad.

15.3. CheckoutStation

Estados:

- Closed;
- Available;
- Busy.

Una estación Busy debe referenciar una entrada Processing. Una estación no busy no debe retener current entry.

15.4. CheckoutTransaction

Campos:

- transaction ID;
- station ID;
- customer ID;
- cart ID;
- line count;
- unit count;
- state;
- failure code.

Estados: Pending, Completed, Failed.

15.5. Límite transaccional

El commit debe coordinar:

1. validar estación y frente de cola;
2. validar sesión y carrito;
3. validar reservas activas;
4. congelar cotización;
5. validar inventario;
6. consumir unidades/reservas;
7. completar carrito/sesión;
8. completar cola y estación;
9. registrar transacción;
10. publicar ingreso en ledger;
11. emitir feedback.

La repetición con el mismo transaction ID o posting key no genera una segunda venta.

15.6. Persistencia

Schema 2 guarda cola, estación y transacciones. Las líneas económicas exactas de una transacción no aparecen en CheckoutTransactionSaveRecord; se guarda line count y unit count. El ledger conserva el importe. Si se requieren devoluciones, auditoría de productos o impuestos por línea, el schema futuro debe guardar CheckoutTransactionLineSaveRecord con precio congelado.

16. Jornada, tiempo y cierre

16.1. StoreDay

Campos:

- StoreDayId;
- StoreDayPolicy;
- state;
- elapsed open seconds.

StoreDayPolicy define duración y AutoBeginClosing.

Estados:

BeforeOpen → Open → Closing → Closed

16.2. Reglas temporales

- elapsed no negativo;
- duración positiva;
- no se acepta cliente fuera de Open;
- el cierre puede iniciarse automáticamente;
- no se llega a Closed hasta cumplir condiciones.

16.3. Snapshot de cierre

StoreClosureSnapshot resume:

- clientes activos;
- entradas de cola;
- estado de estación;
- reservas activas;
- sesiones pendientes.

La transición final solo es válida cuando todos son compatibles con cierre.

16.4. Actividad diaria

StoreDayActivityTracker acumula hechos como:

- llegadas;
- clientes dirigidos a salida;
- checkouts;
- cancelaciones;
- abandonos;
- reservas liberadas.

El resumen puede compararse con ledger y transacciones para detectar discrepancias.

16.5. Persistencia

DayCycleSaveRecord contiene:

- day ID;
- state;
- open duration;
- elapsed open seconds;
- auto begin closing.

Cuando el día está Closed, el snapshot integrado exige:

- clientes despawned;
- cola vacía;
- estación no busy;
- ninguna reserva activa;

- sesiones Abandoned o CheckedOut.
-

17. Economía, precios y ledger

17.1. CurrencyCode

- tres caracteres;
- normalizado a mayúsculas;
- igualdad ordinal;
- no se mezcla con otra moneda.

17.2. Money

Campos:

- MinorUnits: long;
- Currency: CurrencyCode.

Operaciones deben usar checked cuando existe riesgo de overflow. Los valores pueden ser negativos en ledger para gastos si la convención elegida lo permite, pero precios unitarios y cotizaciones deben ser positivos.

17.3. EconomyLedger

Tipos actuales:

- CheckoutRevenue;
- SupplierReceivingCost.

EconomyPostingKey combina tipo y source ID para idempotencia.

Invariantes:

- entry ID único;
- posting key única;
- moneda única;
- day ID válido;
- importe con signo coherente con la convención;
- no modificar entradas confirmadas.

17.4. Saldo

La regla objetivo:

$\text{cash} = \text{openingCash} + \text{sum}(\text{ledger entries affecting cash})$

CashCents puede guardarse como resumen para carga rápida, pero debe validarse o derivarse. No puede evolucionar independientemente del ledger.

17.5. Resultado diario

DailyEconomicResult incluye ingresos de checkout, coste de recepción, resultado bruto, recuentos y métricas de actividad. Es una proyección reproducible. Puede persistirse como histórico cerrado, pero no sustituye a las entradas que lo justifican.

17.6. Extensiones futuras

Nuevos posting types explícitos:

- rent;
- utilities;
- tax;
- salary;
- refund;
- promotion;
- online sale;
- shipping cost;
- publishing milestone;
- royalty settlement;
- infrastructure cost.

No se usarán categorías string libres sin validación.

18. UI, tutorial, accesibilidad y proyecciones

18.1. Proyecciones UI

StoreHudSnapshot y ManagementPanelSnapshot son modelos de lectura. Incluyen día, estado, tiempo, efectivo, clientes, cola, checkout y estado de save.

No deben mutar el dominio. Los commands de UI se envían a servicios de aplicación.

18.2. Navegación UI

UiNavigationState mantiene capas y foco:

- HUD;
- management panel;
- submenu;
- tooltip;
- confirmation;
- pause menu.

Este estado puede ser transitorio. Solo se persiste si existe una decisión de UX clara.

18.3. Tutorial

TutorialProgress usa pasos tipados. Puede persistirse por perfil/partida:

- no iniciado;
- activo;
- completado;
- omitido.

La referencia de ancla visual no debe impedir cargar si la UI cambia; se necesita fallback.

18.4. Accesibilidad

UiAccessibilitySettings contiene escala de UI/texto, reducción de movimiento, duración de mensajes, tutorial y confirmación destructiva. Es preferible persistirlo como configuración de usuario independiente de la partida.

19. Presentación representativa y catálogos visuales

19.1. Separación funcional/visual

`RepresentativePrefabCatalogAsset` enlaza IDs funcionales con prefabs. `Phase1RuntimeAssetRegistryAs` centraliza referencias deterministas.

El mapping visual puede cambiar sin modificar:

- product ID;
- display ID;
- placement definition ID;
- inventory container ID;
- snapshot.

19.2. Catálogo Phase 1

`Phase1FurnitureDefinition` contiene dimensiones, capacidad, coste, flags, material y prefab. `Phase1ProductDefinition` contiene coste mayorista, precio, unidades por caja y referencias visuales.

Es una capa representativa integrada, pero solapa conceptos con `ProductDefinition`, `DisplayDefinition`, catálogos de precios y `supplier catalog`. Debe converger para evitar que un producto tenga precios distintos en dos catálogos sin una regla explícita.

19.3. `Phase1StoreState`

Registra:

- slot/session/generation;
- secuencias de IDs;
- órdenes;
- stocks;
- fixtures colocados;
- ventas completadas;
- ingresos y gastos acumulados.

Debe tratarse como bridge o fachada transitoria. El objetivo no es mantener permanentemente dos modelos de stock, pedidos y economía.

19.4. Regla de authoring

Los assets visuales no contienen progreso. Los campos como `PrefabResourcePath` son referencias de contenido y no deben guardarse en el save. El save conserva el ID funcional y el runtime resuelve la visual vigente.

20. Snapshot integrado schema 2

20.1. Cabecera

`IntegratedGameStateSnapshot.CurrentSchemaVersion = 2.`

Campos de cabecera:

Campo	Regla
<code>SchemaVersion</code>	exactamente 2 en la implementación actual
<code>SessionId</code>	<code>StableId</code> válido

Campo	Regla
SlotId	0..2
CreatedUtc	UTC
UpdatedUtc	UTC y no anterior a creación
CurrentDay	mínimo 1
CashCents	resumen de saldo
CurrencyCode	tres caracteres, mayúsculas

20.2. Subestados

- inventarios;
- órdenes de proveedor;
- displays;
- clientes;
- sesiones de shopping;
- reservas;
- entradas de cola;
- estación;
- transacciones;
- ciclo diario;
- ledger.

20.3. Records persistentes

Record	Campos clave
ProductQuantitySaveRecord	product ID, quantity
InventoryContainerSaveRecord	container ID, capacity, products
SupplierOrderSaveRecord	order/supplier/product, state, ordered/received, cost
DisplaySaveRecord	display/definition/assignment/inventory
CustomerSaveRecord	customer/profile/state/patience/navigation index
ShoppingSessionSaveRecord	customer/intent/cart/state/capacity
ReservationSaveRecord	reservation/customer/cart/display/product/quantity/state
CheckoutQueueEntrySaveRecord	entry/customer/cart/state/position
CheckoutStationSaveRecord	station/state/current entry
CheckoutTransactionSaveRecord	transaction/customer/cart/station/state/counts
DayCycleSaveRecord	day/state/duration/elapsed/auto close
EconomyLedgerSaveRecord	entry/type/source/day/amount/currency

20.4. Validación cruzada

El constructor del snapshot valida:

- IDs únicos de inventario, display, cliente, pedido, reserva, transacción y ledger;
- inventario de cada display existente;
- sesión de shopping con cliente válido;
- unicidad por customer y cart;
- reservas con cliente, display y sesión compatibles;
- cola con posiciones contiguas y relaciones válidas;
- estación busy con entrada processing;
- transacciones con customer, cart y station correctos;
- posting keys únicas;
- day y currency de ledger coherentes;
- reglas adicionales de día cerrado.

20.5. Campos ausentes que requieren decisión

El schema 2 no incluye expresamente:

- objetos colocados/ocupación;
- entregas y cajas detalladas;
- catálogo o manifest version;
- tutorial/settings;
- historial de resultados diarios;
- líneas detalladas de checkout;
- seeds/secuencias de spawn;
- estado de precio configurado por jugador;
- impuestos y costes periódicos;
- unlocks/progresión futura.

No todos deben entrar en schema 3, pero cada uno debe evaluarse contra equivalencia de carga.

21. Serialización, almacenamiento y recuperación

21.1. DTO sin Unity

Los save records no deben contener:

- GameObject;
- Transform;
- MonoBehaviour;
- ScriptableObject;
- Vector3 cuando existe coordenada lógica;
- referencias de asset;
- delegates/eventos;
- servicios.

21.2. Escritura segura

capturar estado coherente
→ construir snapshot
→ validar
→ serializar en memoria
→ escribir temporal
→ flush y cierre
→ conservar backup anterior
→ reemplazar primario de forma segura
→ verificar

21.3. Estados de repositorio

- Success;
- SlotEmpty;
- RecoveredFromBackup;
- ValidationFailure;
- UnsupportedSchema;
- CorruptPrimaryNoBackup;
- StorageFailure.

La UI debe diferenciarlos.

21.4. Recovery

Si el primario falla y el backup es válido:

- cargar backup;
- marcar slot recuperado;
- conservar diagnóstico;
- no ocultar la recuperación;
- evitar sobrescribir inmediatamente el único backup válido.

21.5. Equivalencia de round trip

```
runtime A
→ capture snapshot S1
→ encode JSON
→ decode snapshot S2
→ restore runtime B
→ capture snapshot S3
```

S1 y S3 deben ser equivalentes en estado autoritativo. No se exige igualdad de orden para colecciones donde el orden no es semántico, pero el encoder debería estabilizarlo.

22. Versionado y migraciones

22.1. Regla de schema

El schema cambia cuando cambia la estructura o semántica persistente. No se incrementa solo por renombrar una clase interna que no afecta al JSON.

22.2. Migración incremental

v1 → v2 → v3

Cada paso define:

- versión origen y destino;
- transformación;
- defaults;
- alias de ID;
- validación posterior;
- log de migración;
- fixture de prueba;
- política de fallo.

22.3. Versiones futuras

Un save más nuevo que el ejecutable debe rechazarse como `UnsupportedSchema`, no como corrupción. Un save antiguo sin ruta de migración debe preservarse y mostrar mensaje claro.

22.4. Cambios de catálogo

Cuando una definición desaparece:

1. mantener alias/mapping de migración;
2. reemplazar por definición compatible;
3. convertir a compensación explícita;
4. bloquear carga con diagnóstico si no existe solución segura.

Nunca se inventa silenciosamente un ID.

22.5. Migración de las rutas actuales

Plan recomendado:

1. declarar IntegratedGameStateSnapshot como raíz autoritativa;
 2. integrar placement dinámico;
 3. integrar entregas/cajas cuando sea necesario;
 4. absorber o retirar Phase1StoreState como save independiente;
 5. convertir GameSessionSnapshot en header/fachada o retirarlo;
 6. añadir manifest de contenido;
 7. introducir migrador antes de schema 3;
 8. crear fixtures reales de schema 1 y 2.
-

23. Validación de contenido y authoring

23.1. Validaciones generales

- ID no vacío;
- ID único global dentro de su namespace;
- clave de localización válida;
- definición referenciada existente;
- precio/coste coherente;
- moneda coherente;
- capacidad positiva;
- huella positiva;
- categoría válida;
- tags únicos;
- prefab requerido disponible;
- relación no circular;
- ruta no dependiente del equipo local.

23.2. Productos

- categoría registrada;
- tags registrados;
- precio de venta disponible para productos vendibles;
- al menos un supplier entry si debe poder pedirse;
- visual representativa si entra en el slice.

23.3. Displays

- placement definition válida;
- capacidad coherente;
- visible limit válido;
- categorías permitidas sin duplicados;
- prefab funcional si corresponde.

23.4. Proveedores

- catálogo no vacío para proveedor activo;
- producto existente;
- unidades por caja positivas;
- min/max de cajas coherentes;
- coste no negativo;

- moneda o convención documentada.

23.5. Clientes

- spawn weight no negativo;
- paciencia positiva;
- velocidad positiva;
- targets requeridos en escena;
- profile ID único.

23.6. Herramientas

Las herramientas de Editor deben producir informes reproducibles y fallar antes de Play Mode cuando el contenido del slice sea inválido.

24. Datos derivados, caches y proyecciones

24.1. Derivados principales

Dato	Fuente
capacidad disponible	capacidad - used
unidades visibles	stock display + visible limit
total de pedido	líneas y costes
total de quote	líneas congeladas
longitud de cola	entries activas
saldo	apertura + ledger
resultado diario	ledger + actividad
ocupación de huella	anchor + size + rotation
disponibilidad para reserva	on-hand - reservas activas
estado del slot	resultado repository + header

24.2. Caches

Un registry puede mantener diccionarios por ID. La colección autoritativa debe poder reconstruir el cache y validar duplicados. No se serializan caches privados.

24.3. Proyecciones históricas

Un resumen diario puede persistirse como histórico para gráficos, siempre que:

- tenga day ID;
- indique schema/versión de cálculo si cambia la fórmula;
- no sea usado como saldo autoritativo;
- pueda auditarse contra los hechos retenidos.

25. Idempotencia, concurrencia lógica y límites transaccionales

25.1. Operaciones idempotentes

Deben protegerse mediante IDs o posting keys:

- recepción de caja;
- coste de proveedor;
- checkout;
- ingreso de venta;
- autosave de cierre;
- migración;
- reintento de delivery futura;
- refund futuro.

25.2. No hay concurrencia de threads como excusa

Aunque la mayoría del gameplay se ejecute en main thread, dos eventos, callbacks o reintentos pueden intentar aplicar la misma operación. La idempotencia sigue siendo obligatoria.

25.3. Preflight

El preflight no muta. Debe reunir todos los fallos previsibles antes del commit. Los datos usados para commit deben permanecer válidos; si existe una ventana de cambio, se necesita lock lógico, versión o revalidación.

25.4. Secuencias

Los contadores de Phase 1 generan IDs de orden, fixture y cliente. Si se mantiene esta estrategia, los contadores deben persistirse. No se reutilizan IDs después de eliminar una instancia.

26. Modelos diferidos cercanos

26.1. Empleados

Estado: DIFERIDO CERCANO.

Entidades objetivo:

Entidad	Campos principales
Candidate	candidateId, profileId, salaryRequest, skills, traits, expiry
Employee	employeeId, contractId, level, xp, fatigue, satisfaction, state
EmployeeContract	salary, schedule, startDate, role, termination terms
EmployeeSkill	skillId, level, xp
EmployeeTask	taskId, category, priority, target, reservations, state
SalaryReview	trigger, requestedIncrease, deadline, resolution
LaborReputation	score, events/history

Invariantes:

- un empleado tiene como máximo un contrato activo;
- una tarea no reserva dos veces el mismo recurso;
- el empleado no ejecuta tareas incompatibles simultáneamente;
- salario y pagos generan ledger;
- fatiga y horario usan tiempo de juego, no `Time.time` persistente;
- despedir no elimina historial económico.

26.2. Investigación

Estado: DIFERIDO CERCANO.

```
ResearchNodeDefinition
  nodeId
  branchId
  prerequisites[]
  cost
  duration/progressPolicy
  unlocks[]
```

```
ResearchProgress
  nodeId
  state
  investedAmount
  progress
  startedDay
  completedDay
```

Los unlocks referencian IDs de capacidades o catálogo. No se codifican como booleanos dispersos.

26.3. Puestos informáticos

Estado: DIFERIDO CERCANO.

Entidades objetivo:

- ComputerStationDefinition;
- ComputerStationInstance;
- ComponentDefinition;
- InstalledComponent;
- StationTierResult;
- StationPricingPolicy;
- StationUsageSession;
- MaintenanceState.

El tier de la estación se deriva de componentes. La sesión de uso referencia cliente, estación, inicio, duración, tarifa congelada y resultado. El puesto es una instancia colocada y debe integrarse con placement, energía futura, clientes y ledger.

27. Comercio online y logística

Estado: VISIÓN DE MEDIO PLAZO.

27.1. Entidades

Entidad	Responsabilidad
OnlineCatalogEntry	disponibilidad, precio y visibilidad por canal
OnlineOrder	pedido del cliente online
OnlineOrderLine	producto, cantidad, precio congelado
PickingTask	reserva y recogida desde inventario
Package	contenido, embalaje, peso/fragilidad
Shipment	carrier, tracking lógico, estado y fechas
CarrierDefinition	coste, plazo, fiabilidad
ReturnRequest	motivo, líneas, resolución
Refund	importe, posting y estado

Entidad	Responsabilidad
OnlineCustomerSegment	demanda y expectativas

27.2. Estados de pedido

Created → Paid → Reserved → Picking → Packed → Shipped → Delivered
 ↳ Cancelled
 Delivered → ReturnRequested → Returned → Refunded

27.3. Inventario compartido

El canal online no duplica stock. Usa reservas con ownership/canal explícito. Las reservas online deben coexistir con reservas en tienda mediante una política de prioridad.

27.4. Persistencia

Los pedidos, pagos, paquetes, shipments y refunds son históricos. El tracking visual puede ser derivado, pero los estados y timestamps deben persistirse.

28. Publishing

Estado: VISIÓN EMPRESARIAL TARDÍA.

28.1. Entidades objetivo

- ExternalStudio;
- GameProposal;
- DueDiligenceReport;
- PublishingContract;
- ContractTerm;
- PublishingMilestone;
- EditorialProject;
- MarketingCampaign;
- RoyaltySettlement;
- EditorialRelationship.

28.2. Contratos e hitos

Un contrato debe congelar:

- partes;
- financiación;
- reparto;
- propiedad/licencia;
- hitos;
- fechas;
- obligaciones;
- condiciones de cancelación.

Los hitos y pagos producen ledger. Una renegociación crea una revisión o amendment, no reescribe silenciosamente el contrato histórico.

28.3. Due diligence

El informe es una evaluación fechada con incertidumbre. No debe revelar la verdad absoluta del proyecto; conserva inputs, confidence y conclusiones conocidas en ese momento.

29. Desarrollo interno

Estado: VISIÓN EMPRESARIAL TARDÍA.

Entidades:

- Insight;
- GameBrief;
- DesignPillar;
- Prototype;
- PlaytestReport;
- GreenlightDecision;
- Feature;
- FeatureDependency;
- InternalMilestone;
- InternalBuild;
- Bug;
- ProjectBudget;
- InternalGameRelease.

Reglas:

- una feature tiene estado y coste;
 - dependencias forman un grafo acíclico o se reportan;
 - un build referencia exactamente qué features/bugs incluye;
 - un greenlight es histórico y fechado;
 - reducir alcance no elimina trabajo ya realizado;
 - presupuesto y gastos se integran con ledger;
 - métricas de calidad y mercado son datos separados de la visión del proyecto.
-

30. Plataforma digital

Estado: VISIÓN EMPRESARIAL TARDÍA.

Entidades:

- PlatformUser;
- DigitalLicense;
- PlatformGame;
- StorePage;
- DeveloperAccount;
- DigitalPurchase;
- DigitalRefund;
- Review;
- WishlistEntry;
- PlatformPolicy;
- DeveloperSettlement.

Invariantes:

- una licencia válida referencia usuario y juego;
- una compra completada tiene posting y licencia;

- un refund no se aplica dos veces;
 - una review tiene ownership y política;
 - un settlement congela periodo, ventas, refunds, comisión e importe;
 - la comisión se representa como regla versionada, no como constante dispersa.
-

31. Infraestructura y servicios online

Estado: VISIÓN EMPRESARIAL TARDÍA.

Entidades:

- CapacityPool;
- InfrastructureModule;
- Service;
- ServiceAllocation;
- Incident;
- MaintenanceWindow;
- BackupPolicy;
- Region;
- ProviderContract.

Los pools usan unidades abstractas. Las asignaciones no pueden exceder capacidad. Incidentes y mantenimientos son históricos. La redundancia y backups se modelan como políticas y recursos, no como booleanos aislados.

32. Mercado y competidores

Estado: VISIÓN EMPRESARIAL TARDÍA.

Entidades:

- Competitor;
- MarketSegment;
- StrategyPlan;
- MarketAction;
- Trend;
- MarketIndex;
- CompetitiveRelationship;
- MarketNewsItem.

El competidor no conoce datos ocultos del jugador sin observación. Las acciones tienen momento, objetivo, coste y efecto. Los índices agregados son series temporales. La memoria estratégica se persiste para evitar que la IA olvide decisiones al cargar.

33. Reglas de extensión

33.1. Añadir una entidad

Debe documentarse:

- propósito;
- ID;
- propietario;
- lifecycle;

- estados;
- invariantes;
- relaciones;
- persistencia;
- migración;
- validación;
- tests;
- proyecciones.

33.2. Añadir un campo persistente

Preguntas obligatorias:

1. ¿Es autoritativo o derivado?
2. ¿Quién lo muta?
3. ¿Cuál es el default para saves antiguos?
4. ¿Puede eliminarse/reconstruirse?
5. ¿Necesita ID de catálogo?
6. ¿Qué invariantes cruzadas introduce?
7. ¿Qué fixtures y migraciones requiere?

33.3. Añadir un enum state

Debe revisarse:

- transiciones;
- save/load;
- estados terminales;
- UI;
- cierre de jornada;
- backward compatibility;
- comportamiento ante un valor desconocido.

33.4. No reutilizar campos

Un campo status de pedido no se reutiliza para estado de entrega. Un productId no se convierte en SKU de proveedor. Se añaden conceptos explícitos.

34. Estrategia de pruebas del modelo

34.1. Value objects

- construcción válida/inválida;
- igualdad y hash;
- boundaries;
- overflow;
- serialización.

34.2. Agregados

- transiciones válidas;
- fallos no mutan;
- collections read-only;
- IDs duplicados;
- invariantes de capacidad.

34.3. Servicios multiagregado

- transferencia conserva unidades;
- reserva evita oversell;
- checkout atómico;
- recepción idempotente;
- ledger no duplica posting;
- cierre drena estado.

34.4. Snapshot

- round trip;
- referencias huérfanas;
- posiciones de cola;
- estación incoherente;
- closed day inválido;
- currency mismatch;
- IDs duplicados;
- unsupported schema;
- recovery.

34.5. Catálogos

- IDs duplicados;
- referencias rotas;
- valores inválidos;
- mapping visual ausente;
- orden determinista.

34.6. Migraciones

Cada fixture histórico debe cargarse, migrarse, validarse y volver a guardarse. La migración no se considera probada solo porque compila.

35. Deuda de datos y plan de convergencia

Deuda	Riesgo	Acción
GameSessionSnapshot y snapshot integrado	saldos/días divergentes	unificar raíz o fachada
Phase1StoreState y dominio principal	pedidos/stock duplicados	adaptar y retirar duplicados
placement ausente de schema 2	pérdida de layout	añadir subestado directorio
entregas/cajas no persistidas en detalle	recepción parcial ambigua	records de delivery
transacción sin líneas persistidas	auditoría/refund limitada	añadir líneas cuando sea necesario
tipo de inventario ausente en record	restauración depende de composición	añadir definición/tipo de estado
precios configurables no persistidos	cambios del jugador pueden perderse	definir price state
settings/tutorial mezclables con partida	ownership ambiguo	separar perfil de usuario
arquitectura visual y builders temporales	IDs/mappings paralelos	converger a catálogo
migrador todavía no consolidado	schema futuro bloqueado	implementar antes de migrar

35.1. Orden recomendado

1. Congelar IDs de los seis productos y ocho familias representativas.
2. Definir manifest de contenido.

3. Integrar placement dinámico en snapshot.
 4. Confirmar ownership de cash/day.
 5. Persistir o reconstruir delivery con idempotencia.
 6. Adaptar Phase 1 a los agregados principales.
 7. Implementar migrador y fixtures.
 8. Ejecutar Golden Path con round trip.
 9. Incrementar schema solo después de completar los pasos anteriores.
-

36. Criterios de aceptación del modelo

El modelo se considera apto para cerrar el vertical slice cuando:

1. Todos los IDs del contenido representativo son estables y únicos.
2. Las definiciones no contienen progreso mutable.
3. Las cantidades nunca son negativas.
4. Las transferencias conservan unidades.
5. Las reservas activas no exceden stock disponible.
6. Un carrito solo contiene reservas del mismo cliente.
7. La cola es FIFO y sus posiciones son contiguas.
8. La estación busy referencia una entrada processing.
9. El checkout fallido no muta inventario, reserva, cola, transacción ni dinero.
10. Una venta confirmada se registra una sola vez.
11. Los postings son idempotentes.
12. El día cerrado no conserva actividad pendiente.
13. El saldo y el ledger son coherentes.
14. El snapshot no contiene referencias Unity.
15. Todas las relaciones persistentes se validan al construir/cargar.
16. El round trip conserva estado autoritativo.
17. El backup puede recuperarse y se informa.
18. Un schema futuro se rechaza como no soportado.
19. Los objetos colocados por el jugador sobreviven a guardar/cargar.
20. La arquitectura fija no aparece como mobiliario dinámico.
21. No existen dos rutas de save divergentes por slot.
22. Los catálogos fallan temprano ante contenido inválido.
23. El Golden Path se completa tras carga.
24. Las pruebas de invariantes permanecen verdes.
25. La documentación y trazabilidad están actualizadas.

36.1. Definition of Done de una nueva entidad persistente

- ID y ownership definidos;
- constructor/validator protege invariantes;
- transiciones documentadas;
- record persistente definido;
- capture y restore implementados;
- schema/migración evaluados;
- round trip probado;
- referencias cruzadas probadas;
- UI no se convierte en fuente de verdad;
- logs incluyen ID y causa tipada;
- documentación actualizada.

Anexo A. Inventario de tipos de dominio observados

La tabla se genera desde los archivos C# suministrados. Las propiedades muestran una selección de miembros públicos y no sustituyen la API completa.

Archivo	Tipo	Clase
Access/AccessAnchor.cs	AccessAnchor	clase
Access/AccessAnchorId.cs	AccessAnchorId	estructura
Checkout/CheckoutIdsAndPolicy.cs	CheckoutQueueEntryId	estructura
Checkout/CheckoutIdsAndPolicy.cs	CheckoutStationId	estructura
Checkout/CheckoutIdsAndPolicy.cs	CheckoutTransactionId	estructura
Checkout/CheckoutIdsAndPolicy.cs	CheckoutPolicy	clase
Checkout/CheckoutQueue.cs	CheckoutQueueEntryState	enumeración
Checkout/CheckoutQueue.cs	CheckoutQueueEntry	clase
Checkout/CheckoutQueue.cs	CheckoutQueueFailureReason	enumeración
Checkout/CheckoutQueue.cs	CheckoutQueueResult	clase
Checkout/CheckoutQueue.cs	CheckoutQueue	clase
Checkout/CheckoutStation.cs	CheckoutStationState	enumeración
Checkout/CheckoutStation.cs	CheckoutStation	clase
Checkout/CheckoutTransaction.cs	CheckoutTransactionState	enumeración
Checkout/CheckoutTransaction.cs	CheckoutTransaction	clase
Checkout/CheckoutTransaction.cs	CheckoutTransactionRegistry	clase
Customers/CustomerArrivalClock.cs	CustomerArrivalClock	clase
Customers/CustomerInstance.cs	CustomerInstance	clase
Customers/CustomerInstanceId.cs	CustomerInstanceId	estructura
Customers/CustomerInstanceRegistry.cs	CustomerInstanceRegistry	clase
Customers/CustomerNavigationPlan.cs	CustomerNavigationPlan	clase
Customers/CustomerNavigationPointId.cs	CustomerNavigationPointId	estructura
Customers/CustomerNavigationTarget.cs	CustomerNavigationTarget	clase
Customers/CustomerNavigationTargetType.cs	CustomerNavigationTargetType	enumeración
Customers/CustomerProfile.cs	CustomerProfile	clase
Customers/CustomerProfileId.cs	CustomerProfileId	estructura
Customers/CustomerProfileRegistry.cs	CustomerProfileRegistry	clase
Customers/CustomerSpawnPolicy.cs	CustomerSpawnPolicy	clase
Customers/CustomerSpawnQueue.cs	CustomerSpawnQueue	clase
Customers/CustomerSpawnRequest.cs	CustomerSpawnRequest	clase
Customers/CustomerSpawnRequestId.cs	CustomerSpawnRequestId	estructura
Customers/CustomerState.cs	CustomerState	enumeración
Customers/CustomerTransitionFailureReason.cs	CustomerTransitionFailureReason	enumeración
Customers/CustomerTransitionResult.cs	CustomerTransitionResult	clase
DayCycle/StoreClosureSnapshot.cs	StoreClosureSnapshot	clase
DayCycle/StoreDayActivity.cs	StoreDayActivitySummary	clase
DayCycle/StoreDayActivity.cs	StoreDayActivityTracker	clase
DayCycle/StoreDayCore.cs	StoreDayId	estructura
DayCycle/StoreDayCore.cs	StoreDayState	enumeración
DayCycle/StoreDayCore.cs	StoreDayPolicy	clase
DayCycle/StoreDayCore.cs	StoreDayTransitionFailureReason	enumeración
DayCycle/StoreDayCore.cs	StoreDayTransitionResult	clase
DayCycle/StoreDayCore.cs	StoreDay	clase
Displays/DisplayAssignmentFailureReason.cs	DisplayAssignmentFailureReason	enumeración
Displays/DisplayAssignmentResult.cs	DisplayAssignmentResult	clase
Displays/DisplayClearAssignmentFailureReason.cs	DisplayClearAssignmentFailureReason	enumeración
Displays/DisplayClearAssignmentResult.cs	DisplayClearAssignmentResult	clase
Displays/DisplayDefinition.cs	DisplayDefinition	clase
Displays/DisplayDefinitionId.cs	DisplayDefinitionId	estructura

Archivo	Tipo	Clase
Displays/DisplayDefinitionRegistry.cs	DisplayDefinitionRegistry	clase
Displays/DisplayInstance.cs	DisplayInstance	clase
Displays/DisplayInstanceId.cs	DisplayInstanceId	estructura
Displays/DisplayInstanceRegistry.cs	DisplayInstanceRegistry	clase
Displays/RestockTask.cs	RestockTask	clase
Displays/RestockTaskId.cs	RestockTaskId	estructura
Displays/RestockTaskStatus.cs	RestockTaskStatus	enumeración
Displays/RestockTaskTransitionFailureReason.cs	RestockTaskTransitionFailureReason	enumeración
Displays/RestockTaskTransitionResult.cs	RestockTaskTransitionResult	clase
Economy/DailyEconomicResult.cs	DailyEconomicResult	clase
Economy/EconomyLedger.cs	EconomyLedgerEntryId	estructura
Economy/EconomyLedger.cs	EconomyPostingType	enumeración
Economy/EconomyLedger.cs	EconomyPostingKey	estructura
Economy/EconomyLedger.cs	EconomyLedgerEntry	clase
Economy/EconomyLedger.cs	EconomyLedgerPostFailureReason	enumeración
Economy/EconomyLedger.cs	EconomyLedgerPostResult	clase
Economy/EconomyLedger.cs	EconomyLedger	clase
Economy/Money.cs	CurrencyCode	estructura
Economy/Money.cs	Money	estructura
Economy/ProductSalePricing.cs	ProductSalePrice	clase
Economy/ProductSalePricing.cs	ProductSalePriceCatalog	clase
Economy/ProductSalePricing.cs	CheckoutQuoteLine	clase
Economy/ProductSalePricing.cs	CheckoutQuote	clase
GameSession/GameSession.cs	GameSession	clase
GameSession/GameSessionSnapshot.cs	GameSessionSnapshot	clase
Grid/GridBounds.cs	GridBounds	estructura
Grid/GridCoordinate.cs	GridCoordinate	estructura
Grid/GridFootprint.cs	GridFootprint	clase
Grid/GridRotation.cs	GridRotation	enumeración
Grid/GridRotation.cs	GridRotationExtensions	clase
Grid/GridSize.cs	GridSize	estructura
Identifiers/SaveSlotId.cs	SaveSlotId	estructura
Identifiers/StableId.cs	StableId	estructura
Inventory/InventoryCapacity.cs	InventoryCapacity	estructura
Inventory/InventoryContainer.cs	InventoryContainer	clase
Inventory/InventoryContainerId.cs	InventoryContainerId	estructura
Inventory/InventoryContainerType.cs	InventoryContainerType	enumeración
Inventory/InventoryMutationFailureReason.cs	InventoryMutationFailureReason	enumeración
Inventory/InventoryMutationResult.cs	InventoryMutationResult	clase
Inventory/InventoryStack.cs	InventoryStack	estructura
Inventory/InventoryTransferFailureReason.cs	InventoryTransferFailureReason	enumeración
Inventory/InventoryTransferResult.cs	InventoryTransferResult	clase
Inventory/InventoryTransferService.cs	InventoryTransferService	clase
Inventory/Quantity.cs	Quantity	estructura
Orders/PurchaseOrder.cs	PurchaseOrder	clase
Orders/PurchaseOrderId.cs	PurchaseOrderId	estructura
Orders/PurchaseOrderLine.cs	PurchaseOrderLine	clase
Orders/PurchaseOrderRequestLine.cs	PurchaseOrderRequestLine	estructura
Orders/PurchaseOrderStatus.cs	PurchaseOrderStatus	enumeración

Archivo	Tipo	Clas
Orders/PurchaseOrderTransitionFailureReason.cs	PurchaseOrderTransitionFailureReason	enu
Orders/PurchaseOrderTransitionResult.cs	PurchaseOrderTransitionResult	stru
Persistence/IntegratedGameStateSnapshot.cs	IntegratedGameStateSnapshot	clas
Persistence/SaveRecords.cs	ProductQuantitySaveRecord	clas
Persistence/SaveRecords.cs	InventoryContainerSaveRecord	clas
Persistence/SaveRecords.cs	SupplierOrderSaveRecord	clas
Persistence/SaveRecords.cs	DisplaySaveRecord	clas
Persistence/SaveRecords.cs	CustomerSaveRecord	clas
Persistence/SaveRecords.cs	ShoppingSessionSaveRecord	clas
Persistence/SaveRecords.cs	ReservationSaveRecord	clas
Persistence/SaveRecords.cs	CheckoutQueueEntrySaveRecord	clas
Persistence/SaveRecords.cs	CheckoutStationSaveRecord	clas
Persistence/SaveRecords.cs	CheckoutTransactionSaveRecord	clas
Persistence/SaveRecords.cs	DayCycleSaveRecord	clas
Persistence/SaveRecords.cs	EconomyLedgerSaveRecord	clas
Placement/PlacedObjectRecord.cs	PlacedObjectRecord	clas
Placement/PlacementFailureReason.cs	PlacementFailureReason	enu
Placement/PlacementInstanceId.cs	PlacementInstanceId	stru
Products/ProductCategoryId.cs	ProductCategoryId	stru
Products/ProductDefinition.cs	ProductDefinition	clas
Products/ProductDefinitionId.cs	ProductDefinitionId	stru
Products/ProductDefinitionRegistry.cs	ProductDefinitionRegistry	clas
Products/ProductTagId.cs	ProductTagId	stru
Receiving/Delivery.cs	Delivery	clas
Receiving/DeliveryId.cs	DeliveryId	stru
Receiving/DeliveryMutationFailureReason.cs	DeliveryMutationFailureReason	enu
Receiving/DeliveryMutationResult.cs	DeliveryMutationResult	stru
Receiving/DeliveryStatus.cs	DeliveryStatus	enu
Receiving/ShipmentBox.cs	ShipmentBox	clas
Receiving/ShipmentBoxId.cs	ShipmentBoxId	stru
Shopping/CustomerShoppingSessionRegistry.cs	CustomerShoppingSessionRegistry	clas
Shopping/ShoppingCartAndSession.cs	ShoppingCartMutationFailureReason	enu
Shopping/ShoppingCartAndSession.cs	ShoppingCartMutationResult	clas
Shopping/ShoppingCartAndSession.cs	ShoppingCartLine	clas
Shopping/ShoppingCartAndSession.cs	ShoppingCart	clas
Shopping/ShoppingCartAndSession.cs	CustomerShoppingState	enu
Shopping/ShoppingCartAndSession.cs	CustomerShoppingSession	clas
Shopping/ShoppingIds.cs	ShoppingIntentId	stru
Shopping/ShoppingIds.cs	ShoppingReservationId	stru
Shopping/ShoppingIds.cs	ShoppingCartId	stru
Shopping/ShoppingIntentAndPolicy.cs	ShoppingPolicy	clas
Shopping/ShoppingIntentAndPolicy.cs	ShoppingIntent	clas
Shopping/ShoppingReservations.cs	ShoppingReservationState	enu
Shopping/ShoppingReservations.cs	ShoppingReservation	clas
Shopping/ShoppingReservations.cs	ShoppingReservationRegistry	clas
Suppliers/SupplierCatalog.cs	SupplierCatalog	clas
Suppliers/SupplierCatalogEntry.cs	SupplierCatalogEntry	clas
Suppliers/SupplierCatalogId.cs	SupplierCatalogId	stru
Suppliers/SupplierDefinition.cs	SupplierDefinition	clas

Archivo	Tipo	Clase
Suppliers/SupplierId.cs	SupplierId	struct
UIUX/SlotDescriptor.cs	SlotPresentationState	enum
UIUX/SlotDescriptor.cs	SlotDescriptor	class
UIUX/StoreUiSnapshots.cs	StoreHudSnapshot	class
UIUX/StoreUiSnapshots.cs	ManagementPanelRow	class
UIUX/StoreUiSnapshots.cs	ManagementPanelSnapshot	class
UIUX/TutorialProgress.cs	TutorialStepId	enum
UIUX/TutorialProgress.cs	TutorialProgressState	enum
UIUX/TutorialProgress.cs	TutorialProgress	class
UIUX/TutorialProgress.cs	TutorialBubble	class
UIUX/UiAccessibilitySettings.cs	UiAccessibilitySettings	class
UIUX/UiNavigationState.cs	UiLayerId	enum
UIUX/UiNavigationState.cs	ManagementPanelId	enum
UIUX/UiNavigationState.cs	UiNavigationEntry	class
UIUX/UiNavigationState.cs	UiNavigationState	class
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1FurnitureKind	enum
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1ProductKind	enum
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1CharacterRole	enum
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1OrderState	enum
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1FeedbackKind	enum
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1AudioChannel	enum
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1FurnitureDefinition	class
VerticalSlicePhase1/Phase1CatalogModels.cs	Phase1ProductDefinition	class
VerticalSlicePhase1/Phase1FeedbackEvent.cs	Phase1FeedbackEvent	class
VerticalSlicePhase1/Phase1OperationResult.cs	Phase1OperationStatus	enum
VerticalSlicePhase1/Phase1OperationResult.cs	Phase1OperationResult	class
VerticalSlicePhase1/Phase1State.cs	Phase1OrderRecord	class
VerticalSlicePhase1/Phase1State.cs	Phase1StockRecord	class
VerticalSlicePhase1/Phase1State.cs	Phase1PlacedFixtureRecord	class
VerticalSlicePhase1/Phase1State.cs	Phase1StoreState	class

Anexo B. Estados y enums implementados

Enum	Valores
CheckoutQueueEntryState	Waiting, Called, Processing, Completed, Cancelled
CheckoutQueueFailureReason	None, QueueFull, DuplicateEntryId, CustomerAlreadyQueued
CheckoutStationState	Closed, Available, Busy
CheckoutTransactionState	Pending, Completed, Failed
CustomerNavigationTargetType	Entry, Browse, Exit
CustomerState	WaitingToEnter, Entering, Browsing, Leaving, Despawned
CustomerTransitionFailureReason	None, InvalidState, InvalidElapsedSeconds, AlreadyDespawning
StoreDayState	BeforeOpen, Open, Closing, Closed
StoreDayTransitionFailureReason	None, InvalidState, InvalidElapsedSeconds, ClosingConditionNotMet
DisplayAssignmentFailureReason	None, ProductDefinitionMissing, CategoryNotAllowed, ProductNotAvailable
DisplayClearAssignmentFailureReason	None, NoAssignedProduct, StockRemaining
RestockTaskStatus	Pending, Completed, Cancelled
RestockTaskTransitionFailureReason	None, TaskNotPending, InvalidCompletedQuantity, CompletedWhilePending

Enum	Valores
EconomyPostingType	CheckoutRevenue, SupplierReceivingCost
EconomyLedgerPostFailureReason	None, DuplicateEntryId, DuplicatePosting, CurrencyMismatch
GridRotation	Degrees0, Degrees90, Degrees180, Degrees270
InventoryContainerType	Unspecified, Generic, Storage, Display, Transit
InventoryMutationFailureReason	None, InvalidQuantity, InsufficientQuantity, CapacityExceeded
InventoryTransferFailureReason	None, InvalidQuantity, SameContainer, ProductDefinitionMismatch
PurchaseOrderStatus	Draft, Submitted, Delivered, Received, Cancelled
PurchaseOrderTransitionFailureReason	None, InvalidCurrentStatus
PlacementFailureReason	None, OutOfBounds, Overlap, DuplicateId, NotFound, AccessDenied
DeliveryMutationFailureReason	None, BoxNotFound, BoxAlreadyReceived
DeliveryStatus	AwaitingReceipt, PartiallyReceived, Received
ShoppingCartMutationFailureReason	None, ReservationNotActive, ReservationOwnedByDifferentUser
CustomerShoppingState	Searching, HoldingReservations, ReadyForCheckout, Abandoned
ShoppingReservationState	Active, Released, Consumed
SlotPresentationState	Empty, Ready, Recovered, Corrupt, UnsupportedSchema, Stopped
TutorialStepId	Welcome, MovementAndCamera, OpenManagement, Inventory, Shopping
TutorialProgressState	NotStarted, Active, Completed, Skipped
UiLayerId	Hud, ManagementPanel, Submenu, Tooltip, Confirmation, PauseMenu
ManagementPanelId	None, Inventory, Suppliers, Displays, Customers, ShoppingCart
Phase1FurnitureKind	CheckoutCounter, WallShelf, CentralShelf, LowDisplay, Floor
Phase1ProductKind	PhysicalGame, GameCase, Console, Controller, Headset, Accessories
Phase1CharacterRole	Employee, Customer, Supplier, Player
Phase1OrderState	Ordered, Received, Completed
Phase1FeedbackKind	PlacementValid, PlacementInvalid, ObjectSelected, ObjectDropped
Phase1AudioChannel	Music, Ambience, Ui, Effects
Phase1OperationStatus	Success, NotFound, InvalidState, InsufficientCash, InsufficientStock

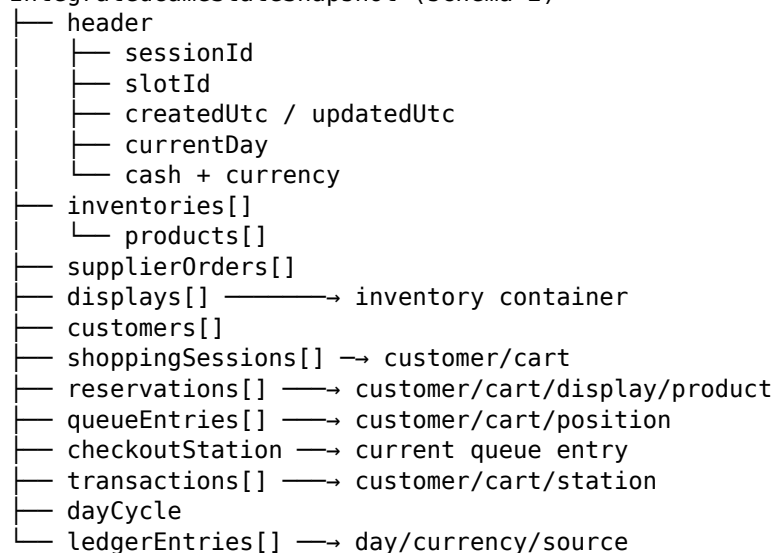
Anexo C. Cardinalidades críticas

Relación	Cardinalidad	Regla
ProductDefinition → InventoryStack	1:N	un producto puede estar en muchos contenedores
InventoryContainer → InventoryStack	1:N	un stack por producto dentro del contenedor
DisplayDefinition → DisplayInstance	1:N	definición compartida
DisplayInstance → InventoryContainer	1:1	contenedor de exposición propio
DisplayInstance → assigned Product	1:0..1	asignación opcional
Supplier → SupplierCatalogEntry	1:N	oferta por producto
PurchaseOrder → PurchaseOrderLine	1:N	al menos una línea
PurchaseOrder → Delivery	1:0..N	permite recepción parcial/futura
Delivery → ShipmentBox	1:N	cajas idempotentes
CustomerProfile → CustomerInstance	1:N	perfil compartido
CustomerInstance → ShoppingSession	1:0..1 activa	ownership único
ShoppingSession → ShoppingCart	1:1	carrito de la sesión
ShoppingCart → Reservation	1:N	líneas basadas en reservas
DisplayInstance → Reservation	1:N	reservas activas limitadas por stock
CheckoutQueue → QueueEntry	1:N ordenada	FIFO
CheckoutStation → current QueueEntry	1:0..1	solo cuando busy
Cart → CheckoutTransaction	1:0..1 completada	evita doble venta

Relación	Cardinalidad	Regla
CheckoutTransaction → Ledger posting	1:1 ingreso	posting idempotente
StoreDay → LedgerEntry	1:N	misma moneda/día
SaveSlot → IntegratedSnapshot	1:0..1 primario + backup	un estado autoritativo

Anexo D. Diagrama de persistencia schema 2

IntegratedGameStateSnapshot (schema 2)



Anexo E. Convenciones de naming

Concepto	Sufijo/patrón
Definición estática	Definition
ID de definición	DefinitionId o ID específico
Instancia mutable	Instance
Registro persistente	SaveRecord
Snapshot raíz	Snapshot
Catálogo	Catalog / Registry según responsabilidad
Política	Policy
Resultado de operación	Result
Causa tipada	FailureReason
Servicio de coordinación	Service / Coordinator
Asset de authoring	Asset
Proyección UI	Snapshot, Descriptor o ViewModel

Anexo F. Glosario

Término	Definición
Agregado	Conjunto con una raíz que protege invariantes internas.
Catálogo	Colección estática e indexada de definiciones.
DTO	Objeto de transporte sin comportamiento de dominio.
Entidad	Objeto con identidad estable durante su ciclo de vida.

Término	Definición
Evento histórico	Hecho confirmado que no se reescribe silenciosamente.
Idempotencia	Repetir una operación identificada no duplica su efecto.
Invariante	Condición que debe cumplirse antes y después de toda transición válida.
Manifest	Lista/versionado del contenido estático compatible con el save.
Ownership	Responsabilidad autoritativa de almacenar y mutar un dato.
Proyección	Vista derivada para lectura, UI o informe.
Record	Representación serializable de una entidad o value object.
Registry	Índice runtime de entidades/definiciones con validación de unicidad.
Snapshot	Captura coherente y versionada de estado persistente.
Value object	Tipo sin identidad propia definido por sus valores.

Actualización del modelo W0

Estado operativo vigente tras W0

Elemento	Estado vigente
Baseline de entrada	main@efbc1a885a1d6819f764ec8cff8a465ad1986661 / aplicación 0.0.26
W0 documental	COMPLETED / DOCUMENTAL PASS
Decisiones funcionales abiertas	0
Sprint17_Phase1	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
H6	APPROVED / IMPLEMENTED / VALIDATED / CLOSED
Vertical Slice	ACCEPTED / IMPLEMENTED / VALIDATED / CLOSED

Entidades y campos requeridos

Agregado / DTO	Campo o relación	Invariante
SimulationClockState	ElapsedBusinessSeconds, SelectedSpeed, IsPaused derivado	Un único rel
StoreDayState	DayNumber, Phase, DayDurationSeconds	300 s config
FundsReservation	ReservationId, OrderIds, AmountMinor, Status	La suma act
DeliveryRun	lista de OrderId, estado y receipt idempotency key	Un Process A
DisplayState	AssignedProductId?, VisibleQuantity, Capacity	Máximo un p
SaveEligibilitySnapshot	fase, mutaciones pendientes, motivo de bloqueo	Guardado m
DailySummary	día, ingresos, costes, impuesto, clientes, ventas	Una fila lógi
ManagementHistory	detalle actual, dos días detallados, lista de summaries, lifetime	No colapsar
WeeklyTaxState	semana, base, importe, posting id	Exactament
PresentationSettings	versión y wall occlusion efectiva false	Preferencia
PlacementAnchor	base-center local y migration marker	Compensaci

La evolución del schema debe aceptar la baseline 0.0.26, inicializar campos nuevos de forma determinista y rechazar estados imposibles sin pérdida silenciosa.

Regla de cierre y no propagación

- W0–W8 están completadas, validadas y cerradas en la baseline 0.0.26.
- Sprint17_Phase1 y W8 están completadas, validadas y cerradas.
- H6 y la Vertical Slice disponen de aceptación propia y están cerrados sobre 0.0.26.

- La Vertical Slice queda cerrada; los sistemas Post-H6 se gestionarán como evolución posterior mediante control de cambios.
 -
-